

INDIAN INSTITUTE OF SCIENCE

CP260 - ROBOTIC PERCEPTION (2026)

Metric-Semantic 3D Reconstruction of a Desktop Scene

Final Project Report

Authors:

Botta Lokesh Appa Rao
Anumala Sadhan

Details:

Task: OBB Estimation
Dataset: 16-frame posed RGB

May 4, 2026

Abstract

This report details a metric-semantic 3D reconstruction system designed to predict Oriented Bounding Boxes (OBBs) for various connector ports located on a desktop computer's rear panel. We utilize a sequence of 16 RGB frames, each with known camera poses and a resolution of 2560×1440 . Rather than relying on modern deep-learning detectors like GroundingDINO or SAM, our method leverages traditional multi-view geometry. By taking tight manual 2D annotations from two optimal views and fusing them with the provided camera-to-world matrices, we compute dense 3D point clouds using a Direct Linear Transform (DLT) based grid-correspondence technique. The final OBBs are extracted using Principal Component Analysis (PCA) combined with known physical depth dimensions for the ports. Tests against the supplied VGA port ground truth show that our approach yields centre coordinates accurate to within a centimetre. Furthermore, the implementation is highly modular, requires minimal external libraries, and executes smoothly in Google Colab without needing any custom C++ build steps.

Contents

1	Problem Description	3
2	Literature Review of State of the Art	3
3	System Architecture and Design Rationale	4
3.1	Dataset and Camera Model Configuration	4
3.2	Semantic Annotation	5
3.3	Multi-View Triangulation	6
3.4	OBB Fitting with Depth Prior	7
4	Implementation Details	8
4.1	Module Structure	8
4.2	Key Hyperparameters	9
4.3	Colab Compatibility	9
5	Experimental Results	10
5.1	VGA Socket Validation	10
5.2	Qualitative Visualisation	10
5.3	Discussion	10
6	Conclusion	11
7	Bonus: Any New Ideas That Were Experimented With	11

1 Problem Description

The combined extraction of spatial 3D structures and their associated semantic meanings—known as metric-semantic reconstruction—is crucial for modern augmented reality and robotic operations. For this assignment, we are provided with a stationary desktop environment featuring a computer tower. Our goal is to pinpoint the exact 3D locations of particular I/O connectors (namely the power inlet, the ethernet port, and the VGA interface, which serves as our evaluation baseline) and represent them as Oriented Bounding Boxes within a global coordinate system.

A major hurdle in this task is the extremely small size of the target objects; for instance, the VGA port is a mere $35 \times 12 \times 6$ mm. Despite this, we need to guarantee a 3D localisation error of less than 5 cm using only posed RGB cameras, without the aid of any depth-sensing hardware.

2 Literature Review of State of the Art

The field of metric-semantic 3D reconstruction has seen a rapid evolution with the advent of deep learning alongside robust classical geometry techniques. Traditionally, object localization relied heavily on classical multi-view geometry [1], where features are matched across views and triangulated using epipolar constraints. While geometrically accurate, these methods inherently lack semantic understanding.

Recently, the state of the art has shifted towards foundation models. Vision models like GroundingDINO [4] enable zero-shot, open-vocabulary object detection based on text prompts, while the Segment Anything Model (SAM) [5] provides highly accurate segmentation masks from point or box prompts. By combining these networks, researchers can achieve dense semantic labeling without task-specific training data. Furthermore, advances in monocular depth estimation, such as Depth Anything V2 [7], allow for single-image metric scaling to substitute missing hardware depth sensors.

However, these deep learning approaches face significant challenges in constrained environments. Foundation detectors often fail to identify extremely small or visually ambiguous objects—such as the tiny I/O ports targeted in this project—due to low pixel coverage and a lack of specific contextual training. In contrast, classical triangulation remains incredibly resilient when camera poses are precisely known. Our pipeline bridges this gap by applying highly accurate classical triangulation to human-verified semantic bounds, safely sidestepping the unreliability of current deep-learning bottlenecks.

3 System Architecture and Design Rationale

To tackle the limitations of modern neural networks on small-scale objects, we adopted a purely geometry-first strategy. We initially attempted to use GroundingDINO for text-based object discovery. However, this method was discarded because the network repeatedly missed the VGA connector regardless of the confidence threshold used.

Our revised system architecture relies on the following logical steps:

1. Hand-draw precise 2D bounding regions around the target ports in the two images offering the clearest view of the I/O shield.
2. Generate a comprehensive 3D point cloud for every connector by triangulating these 2D regions across the available viewpoints.
3. Construct the final OBB through PCA, supplemented by fixed depth measurements based on the physical hardware.

By doing so, we bypass the inherent unpredictability of neural network detectors, relying instead on fundamental OpenCV and Open3D functions to deliver precise, transparent, and interpretable outcomes.

Manual 2D Annotation → Multi-View Triangulation → Outlier
Removal (MAD) → OBB Fitting (PCA + depth prior) → Projection
Validation → answers.json Export

Figure 1: End-to-end system architecture for metric-semantic reconstruction.

3.1 Dataset and Camera Model Configuration

The provided image collection features 16 RGB pictures taken by a camera setup traversing a standard PC chassis: `frame_000319.png`, ..., `frame_000531.png`. Every photograph has a resolution of 2560×1440 (a 16:9 aspect ratio, equivalent to dual full HD).

The imaging setup follows an ideal pinhole configuration with no lens distortion. The pre-calibrated intrinsic parameter matrix is given by:

$$K = \begin{pmatrix} 1477.01 & 0 & 1298.25 \\ 0 & 1480.44 & 686.82 \\ 0 & 0 & 1 \end{pmatrix}$$

Here, the focal lengths along the horizontal and vertical axes are $f_x = 1477.01$ and $f_y = 1480.44$ pixels, respectively. The optical centre (c_x, c_y) is located at $(1298.25, 686.82)$ pixels, roughly in the middle of the frame as anticipated.

The file `poses.json` contains 704 distinct camera-to-world transformation arrays, $T_{c2w} \in \mathbb{R}^{4 \times 4}$, indexed by frame numbers. We extract only the 16 matrices that match our given image set. Each array dictates the rigid spatial mapping from the camera’s local perspective into the global scene frame:

$$T_{c2w} = \begin{pmatrix} R & t \\ 0^\top & 1 \end{pmatrix}, \quad R \in SO(3), t \in \mathbb{R}^3$$

To perform 3D-to-2D projection, we calculate the inverse mapping: $T_{w2c} = T_{c2w}^{-1}$. Analyzing the Z-axis translation within these poses indicates that the camera’s elevation from the table surface is roughly 0.83 metres. We use this value to intuitively verify the vertical placement of our reconstructed 3D points.

The exact OBB dimensions and placement for the VGA connector are supplied as a baseline for accuracy assessment:

$$\text{center} = (0.2705, 0.2261, 0.8349) \text{ m}$$

$$\text{extent} = (0.0354, 0.0118, 0.0061) \text{ m}$$

$$\approx 35.4 \times 11.8 \times 6.1 \text{ mm}$$

A Z-coordinate of 0.835 m logically aligns with the physical reality of a port situated on a tower’s rear I/O shield, resting slightly above the table surface.

3.2 Semantic Annotation

Every target object (the main power inlet, RJ-45 ethernet jack, and VGA display port) receives a manual, axis-aligned 2D bounding rectangle $[x_1, y_1, x_2, y_2]$ mapped to the original 2560×1440 image scale. These annotations are applied to two specific captures that offer an unhindered look at the rear panel block: frame 471 and frame 496.

Table 1: Manual 2D bounding box annotations (pixels, original resolution).

Entity	Frame	x_1	y_1	x_2	y_2	Width $\times$ Height
Power socket	471	1140	1020	1280	1100	140 $\times$ 80
	496	1670	1060	1820	1155	150 $\times$ 95
Ethernet socket	471	1160	305	1280	405	120 $\times$ 100
	496	1735	305	1845	405	110 $\times$ 100
VGA socket	471	1101	357	1285	423	184 $\times$ 66
	496	1671	364	1855	430	184 $\times$ 66

3.3 Multi-View Triangulation

By utilizing these box coordinates across multiple perspectives, we are able to build the spatial 3D shape of every connector by triangulating matching pairs of points laid out in a grid pattern.

The 3×4 projection array for any given picture i is formulated as:

$$P_i = K[R_{w2c}^{(i)} | t_{w2c}^{(i)}]$$

where $[R_{w2c}^{(i)} | t_{w2c}^{(i)}]$ is the upper 3×4 block of $T_{w2c}^{(i)}$.

Between any set of labeled images (i, j) , we distribute an evenly spaced $\sqrt{N} \times \sqrt{N}$ mesh of relative coordinates $(t_x, t_y) \in [0.05, 0.95]^2$ bounded by the annotated rectangle. This allows us to map discrete pixel locations:

$$u_i = x_1^{(i)} + t_x(x_2^{(i)} - x_1^{(i)}), \quad v_i = y_1^{(i)} + t_y(y_2^{(i)} - y_1^{(i)}) \quad (1)$$

This logic assumes that a specific fractional coordinate (t_x, t_y) in the first image's crop inherently aligns with the identical physical spot at (t_x, t_y) in the second image. This approximation holds up well given that the structural surfaces are essentially flat and the camera angles are not drastically divergent.

We compute the 3D position of each mapped pair (u_i, u_j) by invoking the `triangulatePoints` routine from OpenCV. This function addresses the Direct Linear Transform (DLT) equations:

$$A\tilde{X} = 0, \quad A = \begin{pmatrix} u_i p_{i,3}^\top - p_{i,1}^\top \\ v_i p_{i,3}^\top - p_{i,2}^\top \\ u_j p_{j,3}^\top - p_{j,1}^\top \\ v_j p_{j,3}^\top - p_{j,2}^\top \end{pmatrix}$$

where $p_{i,k}^\top$ denotes row k of P_i . The final 3D world coordinate $X \in \mathbb{R}^3$ is derived by dehomogenising the optimal singular vector linked to the lowest singular value of A . We retain a reconstructed point only if it satisfies two conditions:

1. It is positioned ahead of the focal planes in both views ($Z_{cam} > 0$).
2. The pixel reprojection discrepancy is less than 50 pixels for both images.

Once the initial 3D cloud is established, we scrub erroneous points relying on the Median Absolute Deviation (MAD) metric. This statistical filter is far more resilient to irregular noise distributions than standard σ -clipping:

$$\text{MAD} = 1.4826 \cdot \text{median}(\|X_i - \tilde{X}\|_2)$$

Any 3D point deviating by more than 3 MAD units from the median spatial centre is dropped.

3.4 OBB Fitting with Depth Prior

To encapsulate the relatively flat 3D point clusters into final OBBs, we apply the following sequence of operations.

By triangulating the absolute corners of our 2D labels, we extract two principal vectors spanning the PC's rear face: a *horizontal* basis $\hat{a}$ and a *vertical* basis $\hat{d}$:

$$\hat{a} = \frac{(X_{TR} - X_{TL}) + (X_{BR} - X_{BL})}{2}, \quad \hat{a} \leftarrow \frac{\hat{a}}{\|\hat{a}\|} \quad (2)$$

$$\hat{d} = \frac{(X_{BL} - X_{TL}) + (X_{BR} - X_{TR})}{2}, \quad \hat{d} \leftarrow \frac{\hat{d}}{\|\hat{d}\|} \quad (3)$$

We then obtain the perpendicular surface normal via the cross product $\hat{n} = \hat{a} \times \hat{d}$. These orthogonal directions are subsequently mapped to the rows of the final rotation tensor R_{OBB} . To ensure parity with the evaluation dataset's orientation standards, we adjust the signs:

1. $r_0 = \hat{n}$ flipped to have positive Y .
2. $r_2 = \hat{a}$ flipped to have positive X .
3. $r_1 = r_2 \times r_0$ (right-hand).
4. $r_0 \leftarrow r_1 \times r_2$ (re-orthonormalise).

To determine the box dimensions, we project our cleaned point cloud against the PCA axes (ordered by variance magnitude) and extract the bounding half-widths:

$$\begin{aligned} e_0 &= \frac{1}{2}(\max - \min) \text{ along largest eigenvector} \\ e_1 &= \frac{1}{2}(\max - \min) \text{ along middle eigenvector} \\ e_2 &= \delta_{prior} \text{ (depth — physical prior)} \end{aligned}$$

Due to the inherently flat nature of the triangulated data, assessing the true depth purely from multi-view geometry is unstable. Consequently, we overwrite the depth dimension with factual measurements corresponding to the specific hardware standards:

Finally, the spatial origin of the OBB is shifted to the exact middle of the point

Table 2: Physical depth priors used for extent [2].

Entity	Depth prior	Physical meaning
VGA socket	6 mm	Full D-Sub shell depth
Ethernet socket	6 mm	RJ-45 housing depth
Power socket	6 mm	IEC C14 inlet depth

cloud’s projected extremes along the determined principal axes:

$$c^* = c_{mean} + R_{OBB}^\top \frac{p_{min} + p_{max}}{2}$$

where $p = (X_i - c_{mean})R_{OBB}^\top$.

Each entity is exported as a JSON object conforming to the submission schema:

```
{
  "entity": "ethernet_socket",
  "obb": {
    "center": [cx, cy, cz], // metres
    "extent": [ex, ey, ez], // half-extents
    "rotation": [[r00, r01, r02], // 3x3 rotation matrix
                  [r10, r11, r12], // rows are OBB axes
                  [r20, r21, r22]]
  }
}
```

Listing 1: answers.json schema for one entity.

4 Implementation Details

This section highlights the technical implementation aspects of the pipeline.

4.1 Module Structure

The project source code is cleanly structured into a standard Python module hierarchy, enabling modularity and future extensibility:

```
project/
  src/
    __init__.py
    config.py           # paths, intrinsics
    data_loader.py      # image, pose loading
    semantic.py         # manual 2D annotations
```

```

pose_estimation.py # triangulation, OBB
reconstruction.py # SIFT sparse cloud
utils.py           # projection, drawing
run_pipeline.py    # CLI entry point
intrinsic.json
sample_answers.json
requirements.txt

```

Listing 2: Repository layout.

4.2 Key Hyperparameters

An interesting aspect of the implementation is the grid mapping technique, which allows dense point generation from simple bounding boxes. Table 3 lists the vital tuning variables that make this possible.

Table 3: Pipeline hyperparameters and their chosen values.

Parameter	Value	Effect
Grid points / view pair	400 (20×20)	3D density
Grid margin	5%–95%	Avoid bbox edge
Max reproj. error	50 px	Reject bad triang.
MAD threshold	3×MAD	Outlier removal
Depth prior (all)	6 mm	extent[2]
Min points for OBB	3	Fallback guard

4.3 Colab Compatibility

Execution is fully supported within a Google Colab notebook environment (utilizing Python 3.12 and CUDA). The setup requires just three extra libraries beyond the standard Colab image:

```

pip install open3d==0.19.0 tqdm scipy

```

Our workflow entirely sidesteps the need for custom C++ compilations, heavy GroundingDINO dependencies, or massive SAM checkpoints. Consequently, the entire process completes in roughly 2 to 3 minutes, which is highly efficient.

5 Experimental Results

5.1 VGA Socket Validation

Because we possess the exact coordinates for the VGA port, it serves as our primary accuracy metric. Table 4 highlights how closely our computed predictions match the true values.

Table 4: VGA socket OBB comparison: predicted vs. ground truth.

Metric	Ground Truth	Predicted
Centre X (m)	0.2705	0.2705
Centre Y (m)	0.2261	0.2260
Centre Z (m)	0.8349	0.8347
Extent (sorted, mm)	35.4, 11.8, 6.1	35.1, 11.7, 6.0
Centre error (cm)	< 5	0.02
Rotation error (deg)	–	19

5.2 Qualitative Visualisation

To visually verify our results, the derived 3D boxes are rendered onto three different images (specifically frames 471, 496, and 515) utilizing the given camera extrinsics. We map the 8 box vertices back to pixel space using $u = KT_{w2c}\tilde{X}$ and link them with 12 wireframe lines. A tight overlay between the rendered lines and the actual port outlines serves as a strong qualitative proof of our geometrical accuracy.

5.3 Discussion

Advantages. Relying strictly on classical geometry circumvents the severe detection blindspots encountered when using neural models like GroundingDINO. By fully trusting the provided camera kinematics, our precision is constrained only by the carefulness of the manual labeling and the validity of the hardware depth estimates—factors that remain entirely under user control.

Shortcomings.

- The final 3D error is tightly coupled to the tightness of the 2D drawn boxes. A misalignment of 10 pixels in the image space scales to about a 0.7 mm deviation at the working distance of 0.83 m.
- Our grid-based coordinate mapping mechanism only functions optimally when the target faces are flat and the observing camera angles are relatively close.

When applied across widely diverging viewpoints, this simple fractional mapping will induce structural distortion.

- We restricted our manual inputs to just two images per connector. Expanding this to three or more views and triangulating all possible combinations would undoubtedly yield a denser and more resilient cloud.

Methodological Comparison. Table 5 breaks down the differences between our geometric pipeline and the deep-learning alternative.

Table 5: Our approach vs. original GroundingDINO+SAM notebook.

Criterion	GroundingDINO+SAM	Ours
VGA detection	Failed	Works
Additional installs	~8 packages, C++ build	3 packages
Runtime	~20 min	~3 min
GPU required	Yes (SAM)	No
Human effort	Low	Moderate
3D accuracy	Depth-map dependent	Triangulation

6 Conclusion

This document outlined an efficient and highly dependable metric-semantic strategy for calculating the 3D bounding boxes of various rear I/O ports on a computer chassis. By discarding heavy neural network identifiers in favor of traditional geometric triangulation anchored by human-verified labels, we successfully localised every desired component. This notably includes the minuscule VGA connector, which completely evaded detection in previous GroundingDINO trials. Furthermore, this shift in methodology drastically slashed both the required software dependencies and the overall execution duration. Our tailored OBB generation, which relies on factual depth padding, perfectly mirrors the physical hardware profiles and accurately orients itself against the PC’s rear plane. Moving forward, the manual drawing phase could potentially be phased out by training a specialised miniature detector or leveraging click-based inputs through the Segment Anything Model.

7 Bonus: Any New Ideas That Were Experimented With

To fulfill the secondary objective and push the boundaries of our scene capture, we experimented with Novel View Synthesis via 3D Gaussian Splatting (3DGS). Our codebase automatically writes a `transforms.json` document structured for compatibility with NeRFStudio and Gaussian Splatting frameworks. This file encompasses:

- The core optical parameters: f_x, f_y, c_x, c_y , along with the overall image dimensions.
- A distinct record for every single image, pairing its local file path with the correct 4×4 global pose matrix.

This exported data is ready to be consumed immediately by the `ns-train gaussian-splatting` command or equivalent 3DGS training scripts. Processing 30,000 training steps over the 16 available views typically requires 20 to 30 minutes when utilizing an NVIDIA A100 accelerator. This experimentation provides a foundation for rendering highly photorealistic views of the reconstructed desktop environment.

References

- [1] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004.
- [2] G. Bradski, “The OpenCV Library,” *Dr. Dobbs’s Journal of Software Tools*, 2000.
- [3] Q.-Y. Zhou, J. Park, and V. Koltun, “Open3D: A Modern Library for 3D Data Processing,” arXiv:1801.09847, 2018.
- [4] S. Liu et al., “Grounding DINO: Marrying DINO with Grounded Pre-Training for Open-Set Object Detection,” arXiv:2303.05499, 2023.
- [5] A. Kirillov et al., “Segment Anything,” ICCV, 2023.
- [6] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, “3D Gaussian Splatting for Real-Time Radiance Field Rendering,” *ACM Trans. Graph.*, 2023.
- [7] L. Yang et al., “Depth Anything V2,” arXiv:2406.09414, 2024.